

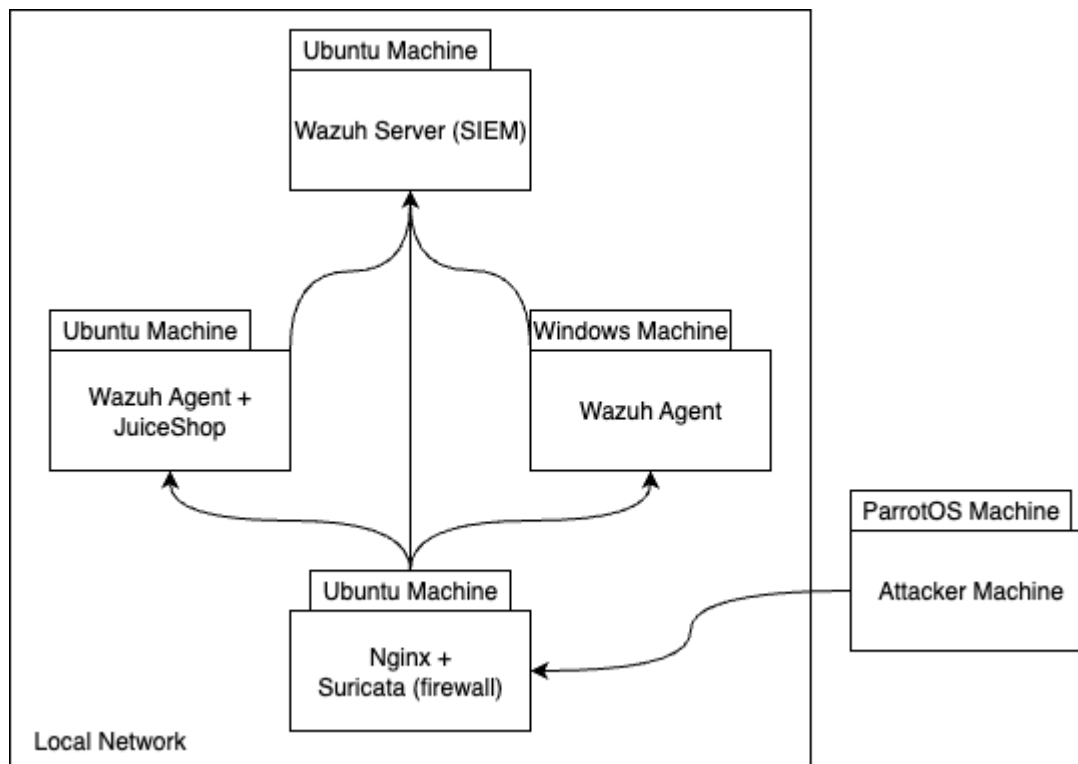
SOC stand simulation project

By:

- Galiev Arsen - a.galiev@innopolis.university
- Nguen Ilya-Linh - i.nguen@innopolis.university
- Zavadskii Peter - p.zavadskii@innopolis.university

Goal/Tasks of the Project

The main goal of the project, is to create a simulated work of protected by a SIEM and WAF network that would be attacked. We created the architecture below:



Therefore, we are able not only detect and prevent typical attacks as brute-force, malware installation and pwns via Wazuh SIEM, but detect and prevent some of them and additional attacks (e.g. injections) via IDS/IPS Suricata.

Thus, we have the following stack :

- 5 virtual machines. 3 Ubuntu, 1 Windows and 1 ParrotOS
- Wazuh (SIEM system)
- Suricata (IDS/IPS)
- Nginx (Load-balancer)

Configuration plan (Environment preparation)

1. Architecture creation, configuring private network using L3 router and starting components deploying.

2. Deploy nginx with custom config that will redirect all the requests to the required servers. Install Suricata and create rules for preventing XSS attacks.
3. Deploy Wazuh master and Wazuh agents on different virtual machines and create connection between them. Deploy JuiceShop application to test attacks.

Development of solution/tests as the PoC

1. We followed the execution plan and created a private network with all necessary components.





Difficulties faced, new skills acquired during the project

There were several difficulties related to Wazuh agent-server connection and Windows machine integration:

- We used following command to set up manager:

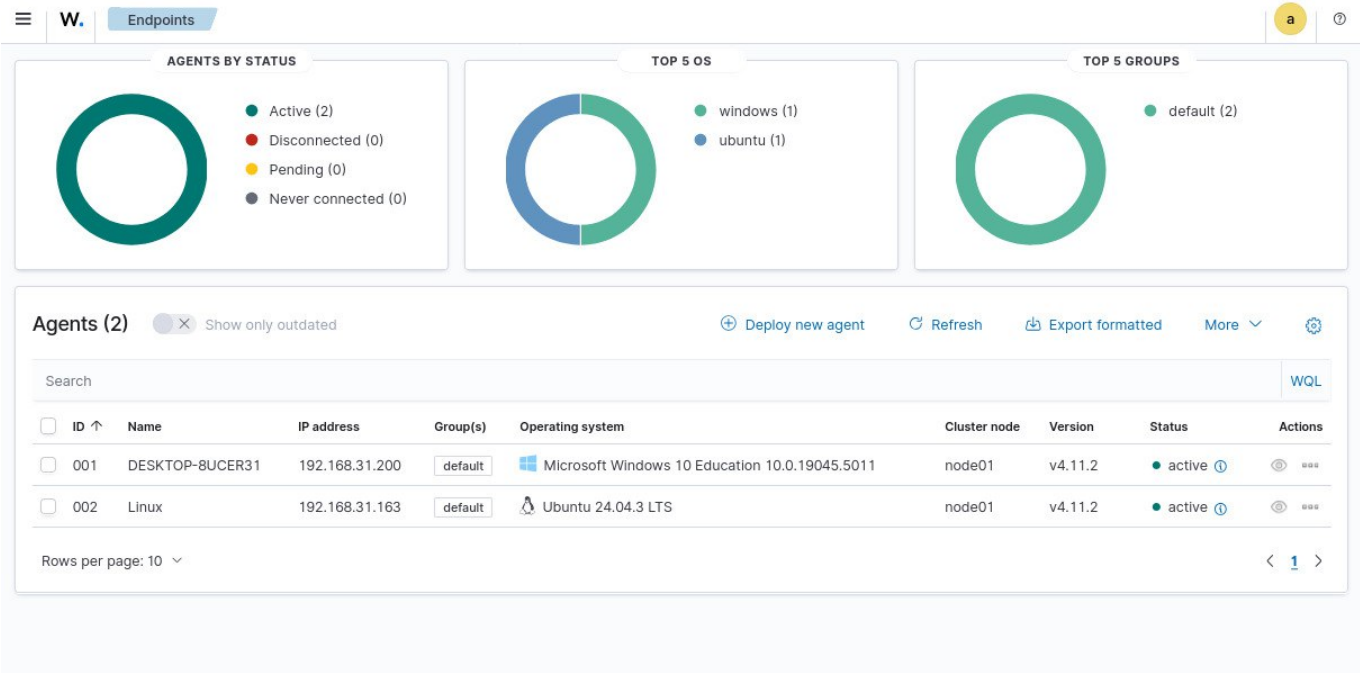
/


```
curl -sO https://packages.wazuh.com/4.11/wazuh-install.sh && sudo bash
./wazuh-install.sh -a
```

And probably since we used not the latest version, the agents configs files were not able to run agents from the box, so we had to remove some unused declarations.

- Config files rules managing

However, we finally coped with it by troubleshooting the machines.



Therefore, we increased our understanding in DevSecOps and SOC field.

Setup

Wazuh

As we stated above, we used a command to install Wazuh manager. After installation and machines availability check, we deployed agents:

- Windwos10

```
Invoke-WebRequest -Uri https://packages.wazuh.com/4.11/windows/wazuh-agent-4.11.0-1.msi -OutFile $env:TMP\wazuh-agent.msi; msixec.exe /i $env:TMP\wazuh-agent.msi /q WAZUH_MANAGER='192.168.31.152' WAZUH_AGENT_NAME='Windows10'; Start-Service wazuhSvc

NET start WazuhSvc
```

- Ubuntu 24.04

```
sudo wget https://packages.wazuh.com/4.11/apt/pool/main/w/wazuh-agent/wazuh-agent_4.11.0-1_amd64.deb && sudo WAZUH_MANAGER='192.168.31.152' dpkg -i ./wazuh-agent_4.11.0-1_amd64.deb

sudo systemctl daemon-reload
sudo systemctl enable wazuh-agent
sudo systemctl start wazuh-agent
```

Since we had conflicts in agents and `ossec.conf` installed by them, we removed rules for *browser extensions, processes and groups*.

Config for Ubuntu: [configs/ubuntu/ossec.conf](#)

Config for Windows: [configs/windows/ossec.conf](#)

Suricata

To install Suricata we used the following guide: [habr.com](#)

```
sudo apt-get update
sudo apt-get install -y software-properties-common
sudo add-apt-repository ppa:oisf/suricata-stable
sudo apt-get update
sudo apt-get install -y suricata
suricata --build-info
sudo suricata -c /etc/suricata/suricata.yaml -i wlp0s20f3 # interface name
```

Nginx

Here how was the load-balancer configured: [nginx.conf](#)

Also, to simulate having DNS-server we used `/etc/hosts` files on other machines:

- Windows: [system32/drivers/etc/hosts](#)
- Linux (Parrot & Ubuntu): [/etc/hosts](#)

PoC, Demonstration

Plan

We have 2 "client machines", one hosting vulnerable application. Our testing included:

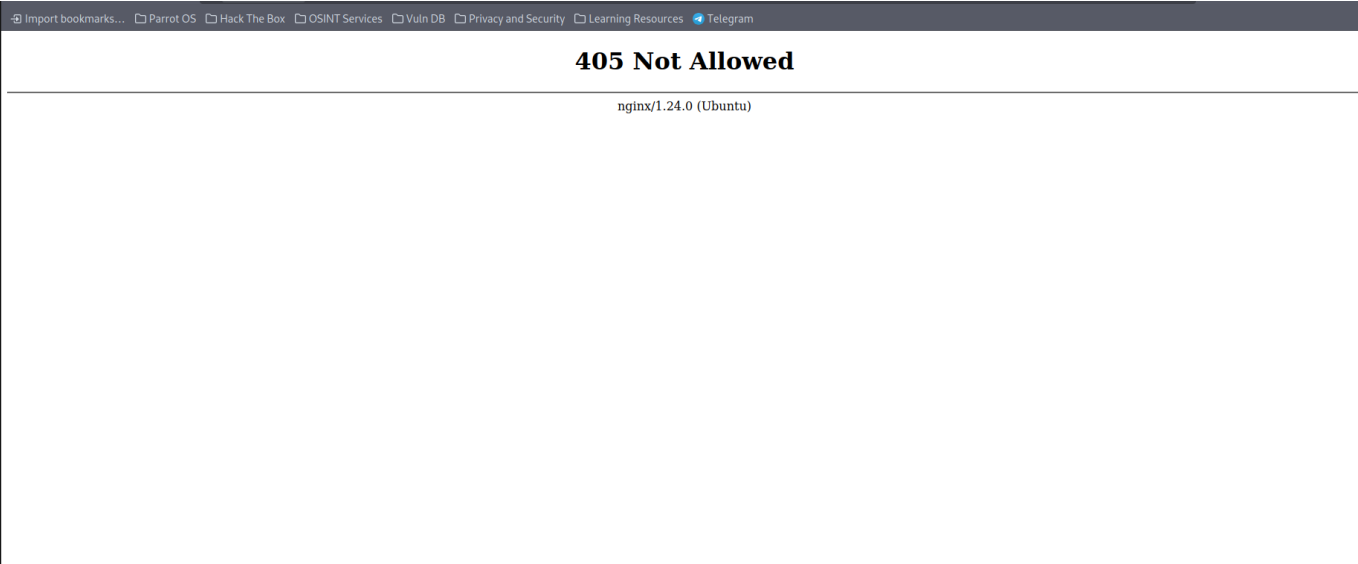
- Attack Juice-Shop via XSSi.
- Ubuntu 24.04 machine attack via `hydra` as we did on lab.
- Windows 10 machine SMB pseudo-brute-force with `smbclient` and `hydra` on ParrotOS.

Juice Shop

Since we created a primitive `anti-XSS suricata rules`, we made XSS attacks simulation (see the demo):

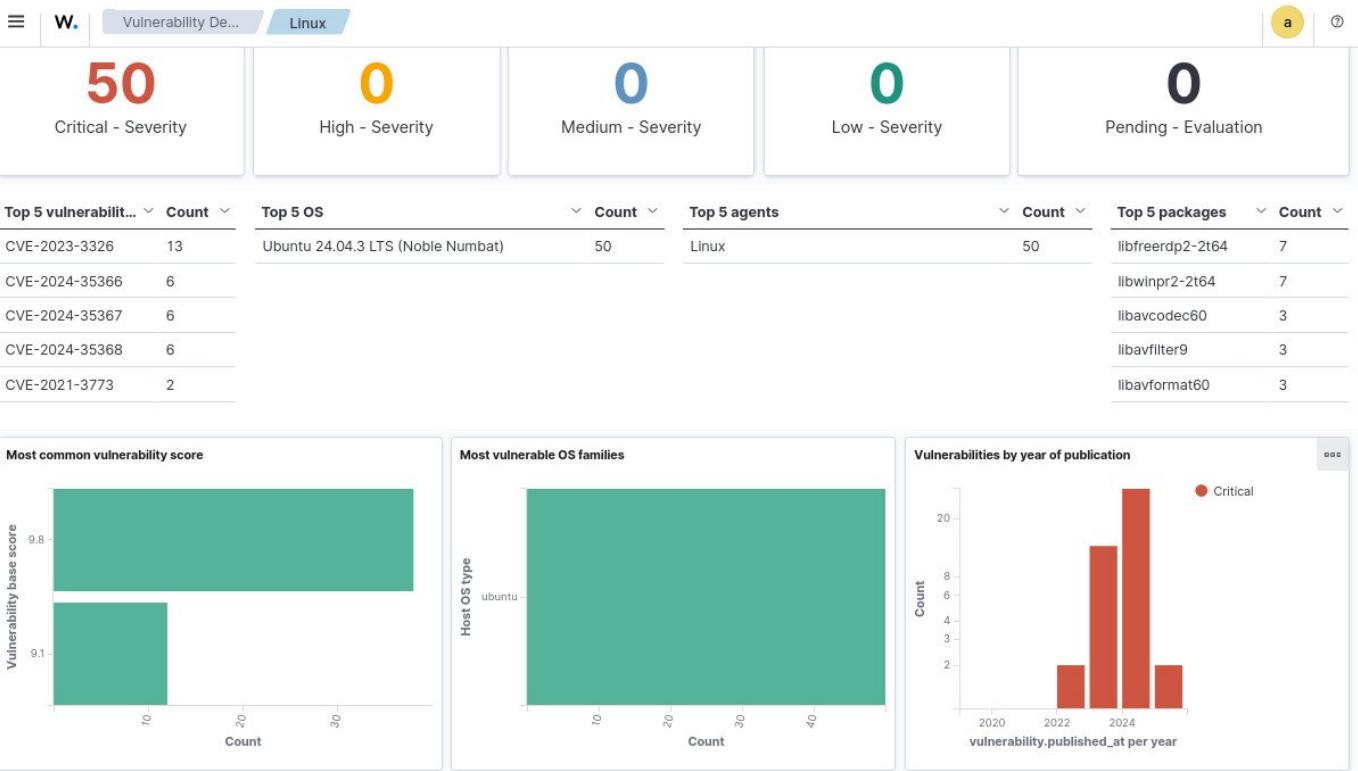
```
curl "http://juiceshop/q?=<iframe%20javascript%3D%22alert('')%2F%3E"
```

and got `405` HTTP response code.

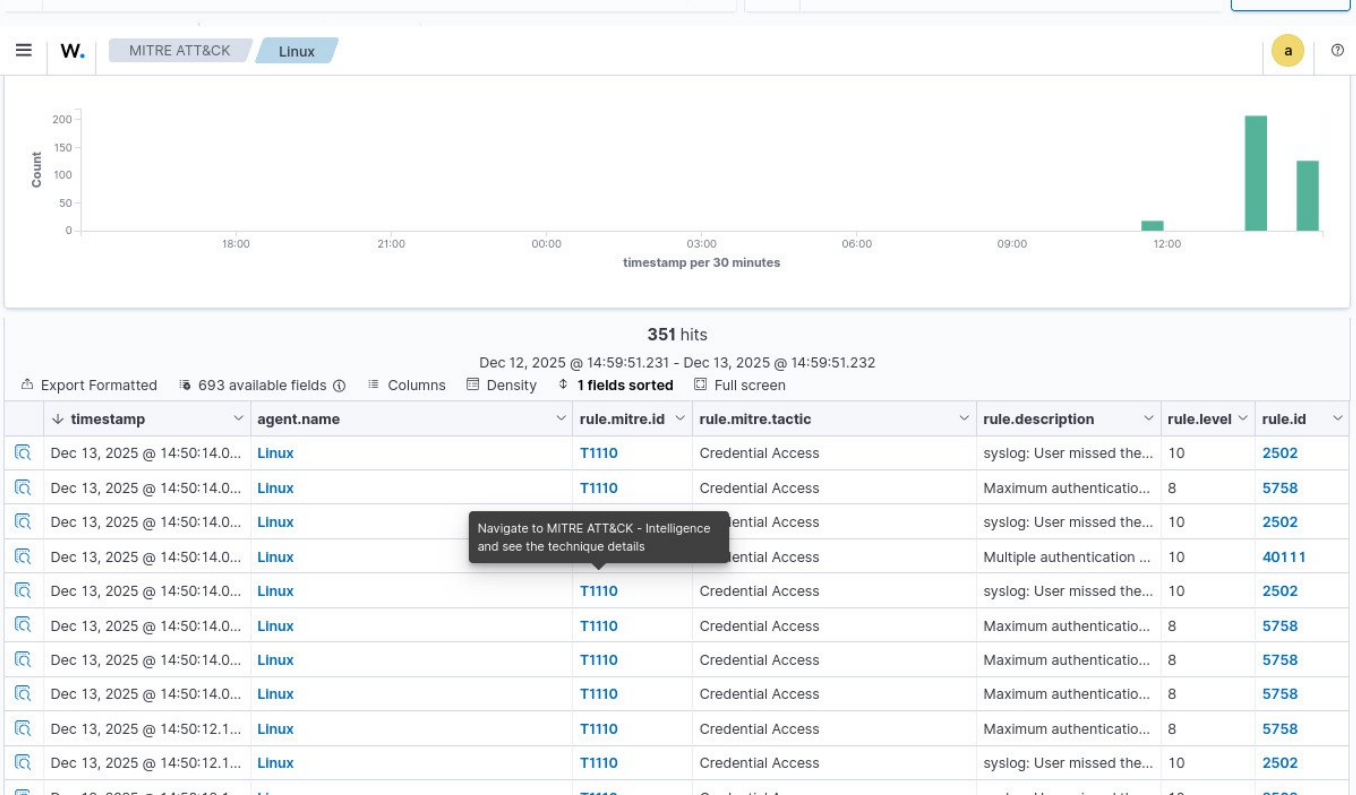


Ubuntu check

Some host information after SCA. Here we filtered only high-severity vulnerabilites since ubuntu host is Arsen's personal workstation:



Logs work well:



Let's repeat the lab scenario with ssh brute force:

Document Details

[View surrounding documents](#)[View single document](#)

Table

JSON

t	_index	wazuh-alerts-4.x-2025.12.13
t	agent.id	002
t	agent.ip	192.168.31.163
t	agent.name	Linux
t	decoder.name	sshd
t	full_log	Dec 13 14:50:12 projacktor-BOM-WXX9 sshd[60752]: Disc onnecting authenticating user root 192.168.31.73 port 36282: Too many authentication failures [preauth]
t	id	1765637414.2747365
t	input.type	log
t	location	journald
t	manager.name	ubuntu-server
t	predecoder.hostname	projacktor-BOM-WXX9
t	predecoder.program_name	sshd
t	predecoder.timestamp	Dec 13 14:50:12
t	rule.description	syslog: User authentication failure.
#	rule.firedtimes	14
t	rule.gdpr	IV_35.7.d, IV_32.2
t	rule.gpg13	7.8

t	predecoder.program_name	sshd
t	predecoder.timestamp	Dec 13 14:50:12
t	rule.description	syslog: User authentication failure.
#	rule.firedtimes	14
t	rule.gdpr	IV_35.7.d, IV_32.2
t	rule.gpg13	7.8
t	rule.groups	syslog, access_control, authentication_failed
t	rule.hipaa	164.312.b
t	rule.id	2501
#	rule.level	5
🔍	rule.mail	false
t	rule.nist_800_53	AU.14, AC.7
t	rule.pci_dss	10.2.4, 10.2.5
t	rule.tsc	CC6.1, CC6.8, CC7.2, CC7.3
📅	timestamp	Dec 13, 2025 @ 14:50:14.079

Again we see familiar attacker's IP, but now in log message.

Windows check

Windows Vulnerability detect:

DashboardInventoryEvents

DESKTOP-8UCER31 (001)

SearchDQLRefresh

wazuh.cluster.name: ubuntu-serveragent.id: 001EvaluatedUnder evaluationAdd filter

W.Vulnerability De...DESKTOP-8UCER31a

2Critical - Severity

137High - Severity

89Medium - Severity

3Low - Severity

0Pending - Evaluation

Top 5 vulnerabilities

	Count
CVE-2021-31693	1
CVE-2021-46873	1
CVE-2022-31693	1
CVE-2023-20867	1
CVE-2023-20900	1

Top 5 OS

	Count
Microsoft Windows 10 Education 10.0.19045.50	231

Top 5 agents

	Count
DESKTOP-8UCER31	231

Top 5 packages

	Count
Microsoft Windows 10	223
VMware Tools	5
WireGuard	2
7-Zip 24.08 (x64)	1

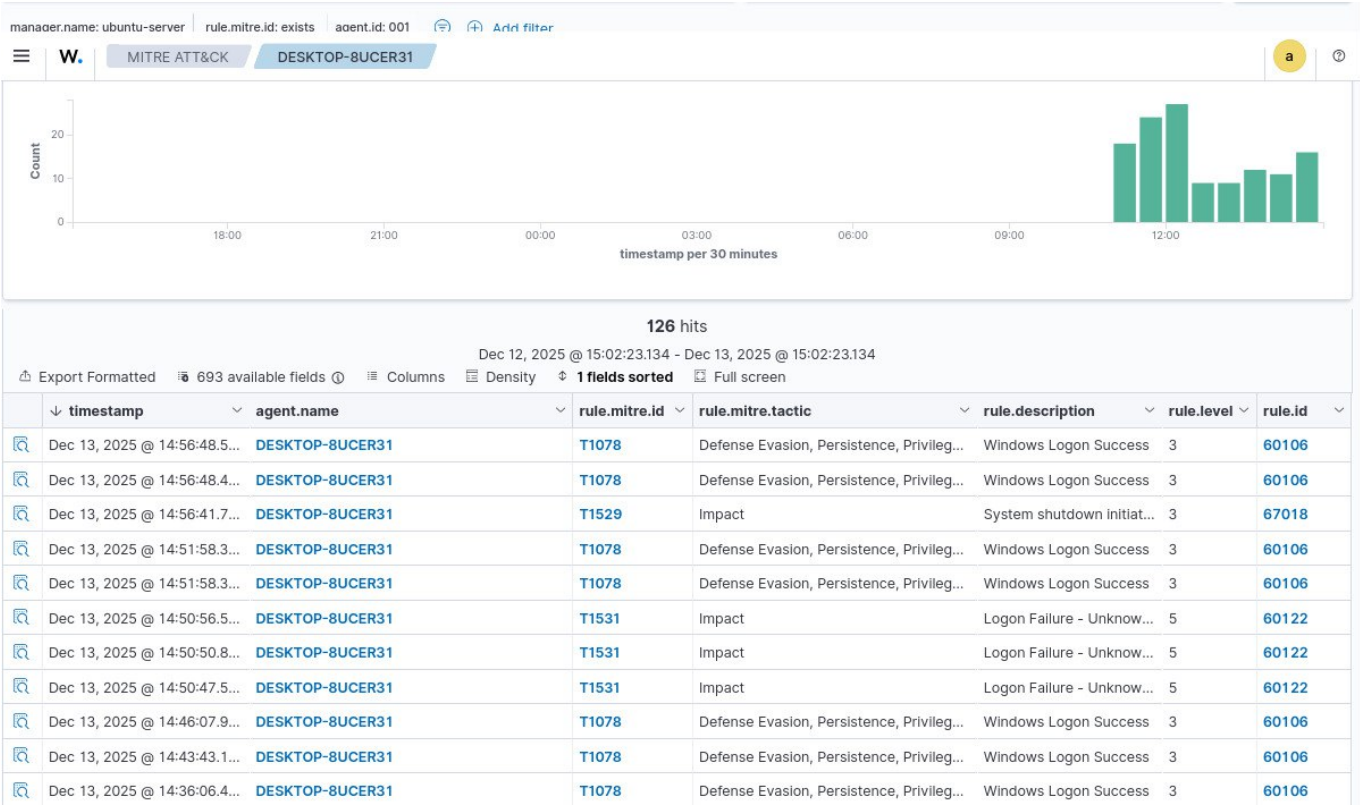
Most common vulnerability score

Most vulnerable OS families

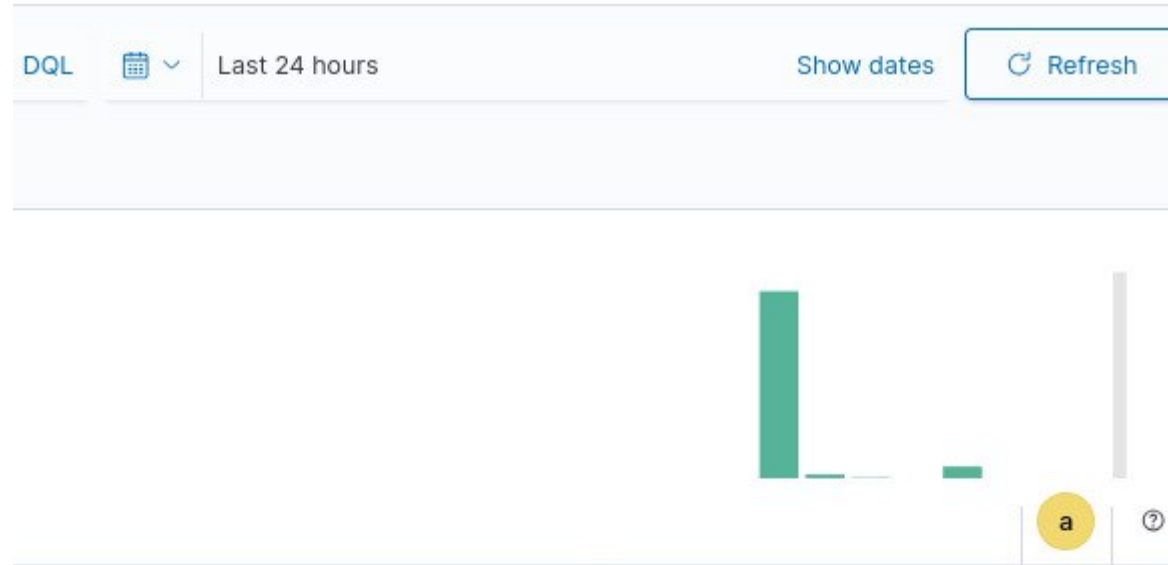
Vulnerabilities by year of publication

/

As you can see on the video, we detected in Wazuh events that Parrot OS attacker's attempts to brute windows guest session password:



Some details and we know that attackers used Parrot OS, it's IP and other details:



Document Details

[View surrounding documents](#)

[View single document](#)

Table

JSON

t _index	wazuh-alerts-4.x-2025.12.13
t agent.id	001
t agent.ip	192.168.31.200
t agent.name	DESKTOP-8UCER31
t data.win.eventdata.authenticationPackageName	NTLM
t data.win.eventdata.failureReason	%%2313
t data.win.eventdata.ipAddress	192.168.31.73
t data.win.eventdata.ipPort	37116
t data.win.eventdata.keyLength	0
t data.win.eventdata.logonProcessName	NtLmSsp
t data.win.eventdata.logonType	3
t data.win.eventdata.processId	0x0
t data.win.eventdata.status	0xc000006d
t data.win.eventdata.subStatus	0xc000006a
t data.win.eventdata.subjectLogonId	0x0
t data.win.eventdata.subjectName	S-1-0-0

Document Details		documents	document
data.win.eventdata.targetUser	guest		
data.win.eventdata.targetUserName			
data.win.eventdata.targetUserSid	S-1-0-0		
data.win.eventdata.workstationName	PARROT		
data.win.system.channel	Security		
data.win.system.computer	DESKTOP-8UCER31		
data.win.system.eventID	4625		
data.win.system.eventRecordID	30894		
data.win.system.keywords	0x8010000000000000		
data.win.system.level	0		
data.win.system.message	"An account failed to log on."		
	Subject:		
	Security ID:	S-1-0-0	
	Account Name:	-	
	Account Domain:	-	
	Logon ID:	0x0	
data.win.system.opcode	0		
data.win.system.processID	636		
data.win.system.providerGuid	{54849625-5478-4994-a5ba-3e3b0328c30d}		
data.win.system.providerName	Microsoft-Windows-Security-Auditing		
data.win.system.severityValue	AUDIT_FAILURE		

Success

During the analysis it gave a thought Windows logs a bit better structured for SIEM normalization, comparing Linux syslog.

Conclusion

In conclusion, this project successfully demonstrated the implementation of a Security Operations Center (SOC) model utilizing open-source tools, including Suricata + Nginx for network intrusion detection and prevention, almost WAF simulation, and Wazuh for endpoint security. The integrated architecture, incorporating Nginx as a load balancer and JuiceShop as a vulnerable application, facilitates comprehensive testing of attack detection mechanisms, such as cross-site scripting (XSS), and incident response capabilities.

The system underscored the critical role of centralized event collection in enabling rapid threat identification and mitigation. Furthermore, this proof-of-concept serves as a foundational platform for advancing more sophisticated security scenarios, encompassing WAF, OpenCTI, SIEM-EDR combinations (Splunk + Wazuh) as cluster-based deployments.

GitHub

<https://github.com/quintet-sdr/demo-soc>

Demo video

<https://github.com/quintet-sdr/demo-soc/blob/main/recordings/GalievNguenZevadskii-demo.mp4>

Tools used

Infra:

- Mi router
- Oracle VirtualBox Community
- VMWare Workstation Pro

Defense tools:

- Wazuh Managment deploy
- Suricata IDS
- Nginx

OS:

- Windows 10 Pro
- Ubuntu 24.04 LTS
- Parrot OS
- Docker
- OWASP Juice Shop

Attack tools:

- smbclient
- hydra
- curl
- nmap